Reporte Técnico de Actividades Práctico-Experimentales Nro. 5

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Josué González
Asignatura	Teoría de la Programación
Ciclo	1A
Unidad	1
Resultado de aprendizaje de la unidad	Identifica los conceptos fundamentales de la teoría de la programación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	005
Título de la Práctica	Del diseño del algoritmo con estructuras secuenciales a la construcción del programa.
Nombre del Docente	José Oswaldo Guamán Quinche
Fecha	Jueves 22 de octubre del 2026 al Viernes 23 de octubre del 2026
Horario	Jueves, 12h30 – 13h30 Viernes, 09h30 - 11h30
Lugar	Aula física asignada al paralelo
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

- Desarrollar la capacidad de transformar un problema en una solución computacional.
- Aplicar estructuras secuenciales en el diseño del algoritmo en un lenguaje de programación.
- Validar la lógica del algoritmo mediante pruebas de escritorio.

3. Materiales, Reactivos, Equipos y Herramientas

- IDE de programación: Visual Studio Code u otro entorno compatible.
- Lenguaje de programación: Según los contenidos de la unidad.

4. Procedimiento / Metodología Ejecutada

● Computador personal con sistema operativo Windows, Linux o macOS.

● Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).

● Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF.

● Conexión a internet estable para acceder a recursos digitales y software en línea.

5. Procedimiento/ Metodología

Metodología de aprendizaje: aprendizaje basado en problemas.

Presentación del objetivo de la práctica.

Contextualización del problema: Un estudiante necesita saber qué calificación debe obtener en el tercer certamen (C3) para aprobar la asignatura con una nota final de 60/100 puntos.

Para calcular la nota final se utilizan las siguientes fórmulas:

Promedio de certámenes (NC)

$$NC = \frac{C1 + C2 + C3}{3}$$

Nota final del ciclo (NF):

$$NF = (NC \cdot 0.7) + (NL \cdot 0.3)$$

C1 y C2 son las notas de los dos primeros certámenes

C3 es la nota del tercer certamen (la que se debe calcular).

NL es la nota de laboratorio.

NF es la nota final de la asignatura.

El programa debe permitir ingresar las notas de C1, C2 y NL; calcular automáticamente la nota mínima necesaria en C3 para que el estudiante apruebe la asignatura.

Nota: Si el resultado es negativo, significa que ya aprueba con las notas actuales.

Desarrollo

La solución del problema se desarrolla mediante la implementación del algoritmo en un lenguaje de programación, empleando exclusivamente estructuras secuenciales. Para ello, seguir los siguientes pasos:

	Elementos	Descripción
Datos de Entrada:	C1	Nota obtenida en el primer certamen.
	C2	Nota obtenida en el segundo certamen.
	NL	Nota obtenida en el laboratorio.
Proceso	Promedio de certámenes (NC)	NC = $\frac{C1+C2+C3}{3}$
	Nota final (NF)	NF = (NC * 0.7) + (NL * 0.3)
	Despeje de C3	NF= 60 y despejo C3 C3 = (((60 - (NL * 0.3)) / 0.7) * 3) - C1 - C2
Datos de Salida	C3 mínimo	Nota mínima necesaria en el tercer certamen para aprobar la asignatura con 60 puntos (puede ser negativa si ya se aprueba con las notas actuales).

Para calcular la nota final se utilizan las siguientes fórmulas:

Promedio de certámenes (NC):

$$NC = \frac{C1 + C2 + C3}{3}$$

Nota final del ciclo (NF):

$$NF = (NC \cdot 0.7) + (NL \cdot 0.3)$$

Diagrama de flujo

Figura [1]



Codificacion en C

```c
#include <stdio.h>

int main() {
    float C1, C2, NL, C3;
    float NF_APROBACION = 60.0;
    float POND_NC = 0.7;
    float POND_NL = 0.3;
    int NUM_CERTAMENES = 3;

    printf("Ingrese la nota del primer certamen (C1): ");
    scanf("%f", &C1);

    printf("Ingrese la nota del segundo certamen (C2): ");
    scanf("%f", &C2);

    printf("Ingrese la nota de laboratorio (NL): ");
    scanf("%f", &NL);

    C3 = (((NF_APROBACION - (NL * POND_NL)) / POND_NC) * NUM_CERTAMENES) - C1 - C2;

    if (C3 < 0) {
        printf("¡Felicidades! Si el resultado es negativo o menor a 0, significa que ya apruebas la asignatura con las notas actuales.\n");
    } else {
        printf("La nota mínima necesaria en el tercer certamen (C3) es: %.2f\n", C3);
    }

    return 0;
}
```

Salida del programa:

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ape5.c -o ape5.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ape5.exe
Ingrese la nota del primer certamen (C1): 55
Ingrese la nota del segundo certamen (C2): 65
Ingrese la nota de laboratorio (NL): 70
La nota m├¡nima necesaria en el tercer certamen (C3) es: 47.14

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Prueba de escritorio

Caso: C1 = 60, C2 = 65, NL = 70

Variable	Operación o valor	Resultado
NF_APROBACION	Constante	60
POND_NC	Constante	0.7
POND_NL	Constante	0.3
NUM_CERTAMENES	Constante	3
C1	Dato ingresado	55
C2	Dato ingresado	65
NL	Dato ingresado	70
C3	(((60 - (70 * 0.3)) / 0.7) * 3) - 55 - 65	47.14

REPORTE TÉCNICO en formato PDF, detallando en la sección de RESULTADOS:

Análisis (datos de entrada, proceso y salida en formato tabla)

Diagrama de flujo (elaborado en una herramienta PSeInt, Lucidchart, Draw.io u otra, con símbolos correctos y una lógica secuencial clara).

Código fuente en lenguaje C (funcional y documentado con comentarios explicativos).

Caso de prueba (1 prueba de escritorio para verificar el funcionamiento)

6. Preguntas de Control:

¿Qué es un programa con estructuras secuenciales?

Un programa con estructuras secuenciales es aquel en el que las instrucciones se ejecutan en un orden estrictamente lineal, una tras otra, desde la primera línea de código hasta la finalización del programa. En este tipo de diseño, el flujo de ejecución avanza secuencialmente sin desviarse por saltos condicionales ni bucles iterativos.

Su desarrollo abarca habitualmente cuatro etapas clave: la definición de variables, el ingreso de datos de entrada, la realización de procesos parciales (cálculos u operaciones) y la visualización de los datos de salida.

¿Cómo se traslada un algoritmo en pseudocódigo o diagrama de flujo a un lenguaje de programación?

El traslado hacia un lenguaje de programación formal corresponde a la fase de codificación, en la cual se traduce la solución lógica estructurada en pseudocódigo o diagrama de flujo hacia las instrucciones de un lenguaje de alto nivel (como C, Java o Python).

Este proceso se realiza mediante los siguientes pasos:

- Traducción sintáctica (Edición): Se escriben las instrucciones adaptándolas a la sintaxis formal del lenguaje, especificando el tipo de dato de cada variable y sustituyendo las órdenes genéricas de entrada, asignación y salida por las sentencias equivalentes del lenguaje.
- Compilación o Interpretación: El código fuente se procesa a través de un compilador o intérprete para detectar y corregir errores de sintaxis.
- Enlace y Ejecución: Se genera el ejecutable correspondiente (como .exe u .out) y se pone en marcha el programa para evaluar su correcto funcionamiento lógico.

¿Por qué es importante validar el programa mediante casos de prueba?

La validación mediante casos de prueba (y pruebas de escritorio) consiste en realizar un seguimiento manual o automatizado paso a paso, simulando la ejecución del programa con valores de entrada específicos.

Es de vital importancia debido a que permite:

- Verificar la exactitud de los resultados: Comprobar que los cálculos matemáticos y lógicos produzcan las respuestas esperadas.
- Detectar y depurar errores semánticos: Identificar fallas lógicas, expresiones mal agrupadas o sentencias fuera de orden antes de la puesta en marcha.
- Garantizar la solidez del programa: Asegurar que la solución sea fiable y cumpla cabalmente con los requerimientos del problema.

7. Evaluación

Para el desarrollo de la presente práctica se utiliza una guía de evaluación, la cual permite valorar de manera objetiva el desempeño de los estudiantes en cada fase de la actividad.

8. Bibliografía

[1] M. M. Arteaga Martínez, Lógica de programación con PSeInt: Enfoque práctico, 1ª ed. Medellín, Colombia: Corporación Universitaria Remington, 2023. [En línea]. Disponible: DOI: 10.22209/9786289585247

[2] J. M. Dueñas Meza, J. J. Valdés Benjumea, y M. C. Rodríguez Niño, Didáctica para el aprendizaje de algoritmos, 1ª ed. Bogotá, Colombia: Sello Editorial UNAD, 2021.

[3] J. E. Guerra Salazar, M. V. Ramos Valencia, y G. E. Vallejo Vallejo, Programando en C desde la práctica: Problemas resueltos, 1ª ed. La Plata, Argentina: Puerto Madero Editorial Académica, 2023. [En línea]. Disponible: ISBN 978-987-82912-8-4

[4] M. Toro Bonilla, Fundamentos de programación: Python, 2ª ed. Sevilla, España: Editorial Universidad de Sevilla, 2021.

9. Anexos:

https://github.com/josue2909/primerciclo.git